COMPUTER SYSTEM DESIGN · VIVA PREPARATION

Bhagiratha

AI-Based Village Pond Planning & Catchment Delineation System
A complete guide to the system, its algorithms, its data layer, and likely viva questions

THE ONE-PARAGRAPH PITCH (MEMORISE THIS)

Bhagiratha is a web app that tells a village administrator **where** to dig a pond and **how big** it should be. The user clicks a point, draws a parcel on a satellite map, or uploads a KML/KMZ contour map. The backend builds an elevation grid (SRTM 30 m from OpenTopography, or a Delaunay interpolation of the contours), fills depressions with **Priority-Flood**, routes water with a latitude-corrected **D8** algorithm, picks the cell where the most water converges, traces its **catchment**, pulls 10 years of rainfall from **Open-Meteo**, estimates runoff with the **Rational Method** $Q = C \cdot I \cdot A$, and sizes a 2.5–4.5 m deep pond holding 70% of Q with a 0–100 suitability score. It is a **modular monolith** (FastAPI) with **PostgreSQL + PostGIS** storage, a rounded-coordinate **cache**, **async** I/O, **admission control** for heavy jobs, and a **fallback chain** + **circuit breaker** for unreliable external APIs. It runs on a 1-CPU / 512 MiB campus container and passed a 446-request stress test with zero errors.

Built from the project source code (app/, scripts/, tests/) and the final report. Every number here comes from the code or from the measured results in the repository.

Contents

1. Problem, requirements and scope
2. High-level architecture
3. Technology stack and why
4. Request lifecycles (the three modes)
5. Algorithms, step by step
6. Worked example with real numbers
7. Database: schema, reads, writes, transactions
8. Caching design
9. Concurrency and asynchronous processing
10. Resilience and error handling
11. API design
12. Deployment and operations
13. Performance and stress testing
14. Testing strategy
15. Design patterns used
16. Weak spots and how you would fix them
17. Scaling to production (10k+ users)
18. Viva questions and answers
19. Numbers cheat sheet

1. Problem, requirements and scope

Problem. Village ponds are often dug where land happens to be free, not where rainwater actually collects. Many stay dry, and some breach in the monsoon. The system recommends a site based on terrain and rainfall instead of guesswork.

Functional requirements

- Show satellite imagery on an interactive map.
- Three input modes: click a point, draw a land area, upload a contour map (KML/KMZ).
- Find the best pond site (maximum water convergence).
- Delineate its catchment and compute area and slope.
- Fetch historical rainfall and compute expected water volume.
- Recommend pond depth, surface area, capacity and a suitability score.
- Overlay results on the map; search villages; view past analyses.

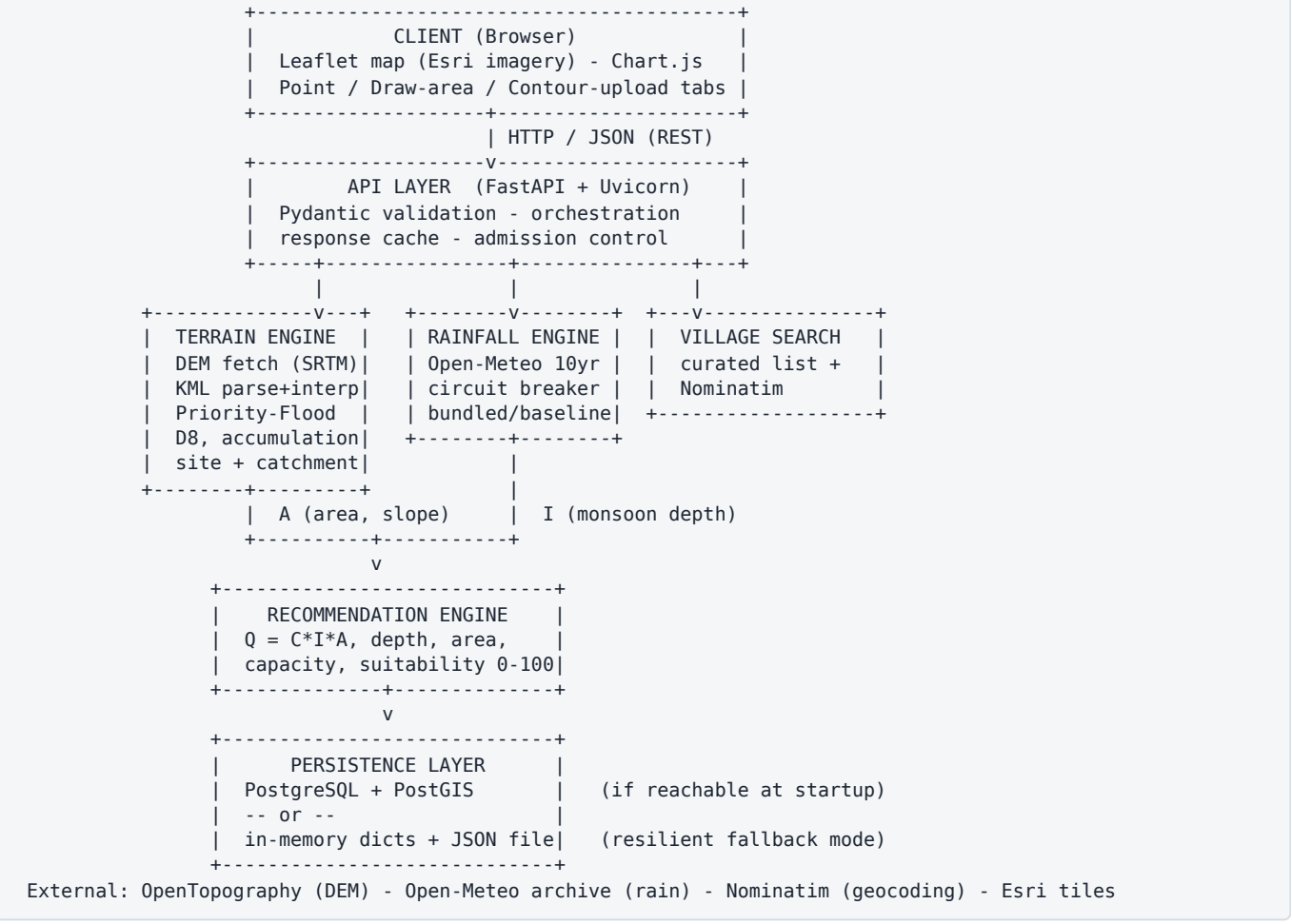
Non-functional requirements

- **Free APIs only** (OpenTopography, Open-Meteo, Nominatim), which are rate-limited and sometimes slow.
- **Tight hardware:** 1 CPU and 512 MiB RAM on the campus container.
- **Availability:** must keep working when an external service is blocked (Open-Meteo is blocked on campus).
- **Latency:** repeat queries fast (cache); heavy work must not freeze the server.
- **Honesty:** every fallback is reported to the user in a `warnings` list.

> **SCOPE LIMIT (SAY THIS BEFORE THEY ASK)**
> "Best site" means **maximum runoff convergence** only. The system does not check land ownership, land use, soil, groundwater, existing rivers/canals, flood safety or cost. On the sample map the chosen point is in the Shivnath river channel. Results are screening recommendations to be verified in the field.

2. High-level architecture

Style: modular monolith. One deployable FastAPI process, internally split into independent modules (Terrain, Rainfall, Recommendation, Village Search, Persistence) that talk only through typed function interfaces (Pydantic models such as `TerrainResult`, `RainfallResult`, `RecommendationResult`).



Why a modular monolith and not microservices?

- **Data size between steps.** Terrain produces large raster arrays (elevation, flow direction, accumulation). Splitting into services would mean serialising and sending these over the network for no gain.
- **One container, one CPU.** Extra services would compete for the same CPU and memory.
- **Team size and timeline.** Single team, about four weeks. One process is simpler to build, test and deploy.
- **Still splittable.** Modules only communicate through typed interfaces, so Terrain could later become its own worker service without changing the Recommendation code.

3. Technology stack and why

Layer	Technology	Reason
Web framework	FastAPI + Uvicorn (ASGI)	Native `async` for parallel external calls; automatic validation and OpenAPI docs at `/docs` from Pydantic models.
Validation / contracts	Pydantic	Rejects bad input at the edge (e.g. `lat` must be in [-90, 90]) with HTTP 422; same models define module interfaces.
Raster I/O	Rasterio (GDAL)	Reads/writes GeoTIFF with a coordinate reference system and affine transform.
Hydrology	PySheds (Numba-compiled)	Tested implementations of Priority-Flood, flat resolution, D8, accumulation, catchment, polygonize.
Interpolation / maths	SciPy `griddata`, NumPy	Delaunay-based linear interpolation of contour points onto a grid; vectorised array maths.
XML parsing	lxml	Fast, namespace-aware parsing of KML; `zipfile` for KMZ.
HTTP client	httpx (async)	Non-blocking calls to OpenTopography, Open-Meteo, Nominatim with timeouts.
Database	PostgreSQL + PostGIS via SQLAlchemy + GeoAlchemy2	Native spatial types (POINT, POLYGON) with GiST spatial indexes; relational integrity with foreign keys.
Frontend	Vanilla JS, Leaflet.js, Chart.js	No build step; Leaflet for map layers and GeoJSON overlays; Chart.js for the rainfall chart.
Ops	Bash start script, watchdog (cron), flock	Reproducible start, automatic restart, single-instance guarantee.

> **WHY POSTGRESQL + POSTGIS AND NOT MONGODB?**
> The core data is spatial (points and catchment polygons). PostGIS stores them as real geometry with spatial indexes and spatial functions (area, containment, distance). The results are also naturally relational: one request has exactly one terrain result, one rainfall result and one recommendation. MongoDB has GeoJSON support but weaker spatial operations and no joins/foreign-key integrity.

> **WHY NOT OPENCV (MENTIONED IN THE BRIEF)?**
> The task is raster hydrology (depression filling, flow routing), not image processing. PySheds implements the standard, published hydrology algorithms directly.

4. Request lifecycles (the three modes)

4.1 Point analysis: `POST /api/analyze {lat, lon, village_id?}`

Validate (Pydantic) → Cache lookup → **hit: return** | Fan-out: terrain ‖ rainfall → Recommend → Write DB / history → Write cache → Respond

1. **Validate.** Pydantic checks lat/lon ranges; invalid body returns 422 automatically.

2. **Read cache.** Key = `"round(lat,3),round(lon,3)"`. If PostgreSQL is up, look in `query_cache` (primary-key lookup) and check the 7-day TTL; otherwise look in the in-memory dict. A hit returns immediately (milliseconds instead of about 18 s).
3. **Fan-out.** Two `asyncio.create_task` calls start the terrain pipeline and the rainfall query **at the same time**. The rainfall query (about 1.25 s) is hidden behind the DEM download (about 12 s).
4. **Terrain.** Download an SRTM tile ±0.05° around the point to a unique temp file; run `delineate_catchment` in a worker thread inside the heavy-job slot; polygonize. If the catchment touches the tile edge, retry once with a ±0.12° tile; if it still touches, add a "lower bound" warning. Failure → HTTP 502 and the rainfall task is cancelled.
5. **Rainfall.** If it fails, use the fallback chain (bundled archive data within 11 km, else regional baseline) and add a warning.
6. **Recommend.** Pure function: `recommend(terrain, rainfall)`.
7. **Write.** Insert into the 4 result tables (or skip if DB down); always add to in-memory history and the JSON history file.
8. **Write cache** only if the result is full quality (no degrading warnings), so a degraded answer is never served from cache later.

4.2 Land-area selection: `POST /api/analyze-area {bounds}`

1. Check `min < max` for both axes (else 422).
2. Download an SRTM tile covering the parcel **plus a margin** (half the parcel size + 0.01°, at least 0.02°), so the upstream catchment is not cut off at the parcel edge. If OpenTopography fails, generate synthetic terrain and add a strong warning ("do not use for site decisions").
3. Run `find_pond_site` with `search_bounds` = the parcel: flow is routed over the whole larger tile, but the chosen cell must lie **inside** the parcel.
4. Rainfall for the chosen site, recommendation, record in history, respond.

4.3 Contour upload: `POST /analyzeContour` (multipart file)

1. Reject anything that is not `.kml` / `.kmz` (400).
2. Run the whole CPU-heavy pipeline as **one job** in the heavy-job slot (`_contour_job`): parse → interpolate to grid → write temporary GeoTIFF → Priority-Flood/D8/accumulation → pick site → trace catchment → polygonize. Large arrays are deleted early to save memory.
3. Error mapping: parse error → 400; cannot build a grid → 422; queue timeout → 503 + `Retry-After: 15`; anything else → 500.
4. Add warnings if the catchment touches the map edge (area is a lower bound) or is zero.
5. Rainfall for the site, recommendation, record, respond.

> **KEY DESIGN IDEA: ADAPTER**
> Both elevation sources (SRTM download and interpolated contours) are converted to the **same float32 GeoTIFF**. The hydrology pipeline does not know or care which source produced it. One tested pipeline, two inputs.

5. Algorithms, step by step

5.1 KML/KMZ parsing (`kml_parser.py`)

- KMZ is a zip: open with `zipfile`, read the first `.kml` entry. Bad zip → `ContourParseError`.
- Parse XML with lxml; find every `<Placemark>` (with or without the KML namespace).

- **Elevation** comes from the `<name>` (e.g. "277.0"), or a regex finds a number in labels like "Contour 280m", or else the mean of the coordinates' Z values.
- **Coordinates** are "lon,lat[,alt]" tokens in the LineString. Keep contours with an elevation and at least 2 points.
- Sample file: 1,355 contours, elevations 267–298 m.

5.2 Contour → DEM interpolation (`dem_from_contours.py`)

1. Every vertex of every contour becomes a scattered sample (lon, lat, elevation).
2. **Decimation:** if more than 20,000 points, keep every *k*-th point, $k = \lfloor n / 20{,}000 \rfloor$. Deterministic and spatially even (not random). Sample: 159,113 → 22,731 points. Reason: Delaunay cost in time and memory grows with point count, and this was the biggest memory consumer.
3. **Grid:** 200 cells along the longer axis, proportional along the shorter (sample: 152 × 200). Rows go north → south (raster convention).
4. **Linear interpolation** with `scipy.interpolate.griddata`: builds a **Delaunay triangulation** of the points (Qhull, O(n log n)) and interpolates each grid cell from the three corners of its triangle (barycentric weights). Linear is chosen over cubic because cubic overshoots and creates fake pits/peaks.
5. Cells outside the convex hull come back NaN → filled with **nearest-neighbour** values, because flow routing needs a complete grid.
6. Write a GeoTIFF (EPSG:4326) whose **pixel centres** coincide with the sample positions (origin shifted by half a pixel). A test checks this.

5.3 DEM conditioning: Priority-Flood (Barnes et al., 2014)

Why: Real DEMs contain small pits (noise). Water in D8 would get stuck in a pit and never reach the outlet, breaking the drainage network.

```
PriorityFlood(DEM):
    PQ  = min-priority queue;  closed = all False
    push every border cell into PQ (priority = elevation); mark closed
    while PQ not empty:
        c = PQ.pop()                      # lowest cell on the "flood front"
        for each neighbour n of c (8-connected) not closed:
            DEM[n] = max(DEM[n], DEM[c])  # raise pit up to its spill level
            closed[n] = True
            PQ.push(n)
```

- Floods inward from the edges like rising water; any cell lower than the cell it was reached from is raised to that level.
- **Complexity:** O(N log N) for N cells (each cell pushed/popped once from a heap).
- **Then `resolve_flats`:** filling creates perfectly flat areas where D8 has no downhill neighbour. Flats get a tiny artificial gradient (away from higher ground, towards lower ground) so every cell has a defined flow direction.

5.4 D8 flow direction (O'Callaghan & Mark, 1984), latitude-corrected

Each cell drains to **exactly one** of its 8 neighbours: the one with the steepest downhill slope.

$$\text{direction}(i,j) = \text{argmax}_{k \in 8\ \text{neighbours}}\ (z_{i,j} - z_k) / d_k$$

d_k is Δx, Δy, or the diagonal $\sqrt{\Delta x^2+\Delta y^2}$. **The latitude correction (a point you should mention):** the raster is in degrees, but one degree of longitude is shorter than one degree of latitude by a factor cos(φ). The code builds a separate "metric" grid for flow routing with:

$$\Delta x = \Delta\lambda \cdot 111{,}320 \cdot \cos\varphi \qquad \Delta y = \Delta\varphi \cdot 111{,}320 \quad \text{(metres)}$$

Without this, east–west slopes are misjudged by about 7% at 21°N (cos 21° ≈ 0.934). O(N).

5.5 Flow accumulation

For each cell: the number of cells upstream that drain through it (including itself). D8 directions form a **forest (tree) of flow paths**; accumulation is computed by processing cells in topological order (sources first) and adding each cell's count to its downstream cell. O(N). High accumulation = channels/valleys.

5.6 Pond-site selection

- **Contour and area modes (automatic):** pick the cell with the **maximum accumulation** in the interior. An **8% border** on each side is excluded because accumulation near a DEM edge is distorted (flow "runs off" the edge). In area mode the search is further restricted to cells whose centres are inside the drawn parcel (if they overlap the interior; otherwise fall back to the plain interior).
- **Point mode (pour-point snapping):** a click is rarely exactly on the one-cell-wide D8 channel, and a point one cell off traces only a tiny hillslope. The point is moved to the highest-accumulation cell within a 5×5 window (radius 2 cells ≈ 60 m on SRTM). This is the standard GIS "Snap Pour Point" step.

5.7 Catchment delineation and polygonization

- Starting from the pour cell, walk **upstream**: include every neighbour whose D8 direction points into the current cell, recursively (a graph traversal, BFS/DFS). Result: a boolean mask. Cost O(size of catchment).
- **Area** = number of cells × metric cell area (Δx·Δy), in km².
- **Average slope:** `np.gradient` (central differences) gives ∂z/∂x and ∂z/∂y in metres; slope% = 100·√((∂z/∂x)² + (∂z/∂y)²); mean over catchment cells.
- **Edge check:** if the mask touches any raster border, the true catchment may continue outside, so the area is a lower bound (point mode retries with a bigger tile).
- **Polygonize:** convert the mask to GeoJSON Polygon (or MultiPolygon) in lon/lat for Leaflet. Consistency check: polygon area 3.9114 km² vs cell-count area 3.9116 km².

5.8 Rainfall aggregation

- Query Open-Meteo archive for daily `precipitation_sum` for the last 10 complete years (current year − 10 to last year).
- Sum monsoon (June–September) and non-monsoon days separately, skip `null` days, divide by the number of distinct years.
- Output: annual average mm, monsoon mm, non-monsoon mm, data_years.

5.9 Runoff and pond sizing (`recommendation.py`)

$$Q = C \cdot I \cdot A \qquad C = 0.35,\ \ I = \text{mean monsoon depth (m)},\ \ A = \text{catchment area (m}^2\text{)}$$

- This is the **volumetric** form of the Rational Method: Q is a seasonal **volume** (m³), not the peak discharge (m³/s) of the classical formula. C = 0.35 is a textbook coefficient for cultivated loamy land (tables give about 0.2–0.5).
- **Capacity** V = 0.70 · Q (target capture; the rest spills).
- **Depth:** 2.5 m if V ≤ 5,000 m³ (shallower ponds lose too much to evaporation); 4.5 m if V ≥ 200,000 m³ (practical limit for unlined earthen banks); linear in between: d = 2.5 + 2·(V − 5000)/195000.
- **Surface area** = V / d.

Suitability score (0–100)

$$S = 0.40 \cdot S_{catchment} + 0.35 \cdot S_{slope} + 0.25 \cdot S_{rainfall}, \text{ clamped to } [5, 98]$$

Component	Rule	Reason
S_{slope}	100 if slope ≤ 4%; else max(10, 100 − 12·(slope − 4))	Gentle slopes need less earthwork and have lower breach risk.
$S_{catchment}$	<0.1 km²: 10→50; 0.1–0.5: 50→100; 0.5–5: 100; >5: −3 per km², floor 50. Piecewise-linear and **continuous** (no jumps).	Too small: pond won't fill. Too large: overflow / spillway risk.
$S_{rainfall}$	≥1000 mm: 100; 500–1000: 60→100; <500: max(15, 60·mm/500)	Reliability of filling.

Why clamp at 98? A screening tool should never claim 100% certainty, since many factors (soil, ownership, groundwater) are not modelled.

5.10 Complexity summary

Step	Time	Measured (sample map)
KML parse	O(total vertices)	0.33 s
Delaunay triangulation + linear interp	O(n log n) + O(grid cells · log n)	0.72 s
Priority-Flood	O(N log N)	0.16 s total (≈3×10⁴ cells)
Resolve flats, D8, accumulation	O(N)	
Site search (argmax)	O(N)	
Catchment trace	O(catchment cells)	
Recommendation	O(1)	negligible

6. Worked example with real numbers (sample contour map)

Quantity	Value
Contours parsed / elevation range	1,355 lines, 267–298 m, area ≈ 3.2 × 2.6 km near Durg
Suggested site	21.24171° N, 81.28692° E
Catchment area A / mean slope	3.9116 km² / 3.99%
Rainfall (10 yr)	1,415.2 mm annual, 1,251.4 mm monsoon
Runoff Q	0.35 × 1.2514 m × 3,911,600 m² = **1,713,242 m³**
Capacity V = 0.7 Q	1,199,269 m³ (≥ 200,000 → depth 4.5 m)
Surface area V / d	1,199,269 / 4.5 = 266,504 m²
Score	catchment 100 (in 0.5–5 band), slope 100 (≤4%), rainfall 100 (≥1000) → raw 100 → clamped **98**
Total time	under 1 s on the 1-CPU server (after thread pinning)

Land-area example: parcel 21.235–21.255° N, 81.278–81.298° E on real SRTM: pond inside the parcel at 21.23611° N, 81.27806° E, catchment 7.089 km² extending upstream outside the parcel, runoff 3,104,911 m³.

> The hand-calculated Q matches the system output exactly. With no field measurements available, correctness was checked by **consistency**: polygon area vs cell-count area, valid polygon, not touching the map edge, and formulas verified by unit tests.

7. Database: schema, reads, writes, transactions

7.1 Two storage modes, chosen at startup

- On startup, `is_db_available()` runs `SELECT 1` with a 3-second connect timeout.
- **DB mode** (PostgreSQL reachable): `Base.metadata.create_all()` creates the tables if missing; reads and writes go to PostGIS.
- **Resilient file mode** (DB unreachable, as on the evaluation container): history in an in-memory list (max 100) persisted to `data/analyses_history.json`; cache in an in-memory dict (max 500 entries). Every endpoint still works.
- The DB session is injected with FastAPI's `Depends(_get_db_lazy)`, which yields a session, or `None` in file mode. One session per request, always closed in `finally`.

7.2 Schema (6 tables, SRID 4326 = WGS84 lat/lon)

```
village                      analysis_request                  terrain_result            (1:1)
------------------           ------------------------          -------------------------
id        PK  int   <----+   id          PK int  <------+-------- request_id PK,FK
name          text       +--- village_id  FK (nullable)  |         catchment_polygon GEOMETRY(4326)
centroid  POINT(4326)        point       POINT(4326)    |         area_km2   float
                             created_at  timestamptz    |         avg_slope  float
                                         default now()  |
                                                        |        rainfall_result            (1:1)
                                                        +-------- request_id PK,FK
                                                        |         annual_avg_mm float
                                                        |         seasonal_json JSON
                                                        |         data_years    int
                                                        |
                                                        |        pond_recommendation        (1:1)
                                                        +-------- request_id PK,FK
                                                                  runoff_m3, depth_m, surface_area_m2,
                                                                  capacity_m3, suitability_score

query_cache  (standalone)
------------------------------
cache_key     PK text   e.g. "21.218,81.339"
response_json JSON      the full AnalyzeResponse
created_at    timestamptz default now()
```

- **Relationships:** village 1 → N analysis_request; analysis_request 1 → 1 each result table. The 1:1 is enforced by making `request_id` both the **primary key and the foreign key** of each result table (a "shared primary key").
- **Why split into separate result tables** instead of one wide row? Each module owns its own table (matches the module boundaries); rainfall can fail while terrain succeeds, so rows can be absent instead of full of NULLs; easier to evolve each part.
- **Normalisation:** roughly 3NF. `seasonal_json` is a deliberate small denormalisation (a JSON blob of monsoon/non-monsoon) because it is always read together and never queried by field.
- **Geometry:** GeoAlchemy2 `Geometry` columns. `catchment_polygon` uses the generic GEOMETRY type because it can be a Polygon or a MultiPolygon. Points are stored as `Point(lon, lat)` (x = lon, the GIS convention).

7.3 Indexes

- **Primary-key B-tree indexes** on every `id`, `request_id` and `cache_key`. The cache lookup and "get analysis by id" are single index lookups: O(log n).
- **GiST spatial indexes**: GeoAlchemy2 creates one by default on each geometry column (`point`, `centroid`, `catchment_polygon`). They make spatial queries like "analyses within 5 km" or "catchments containing this point" fast (R-tree-like bounding-box search).

- **Not indexed (improvement):** `analysis_request.created_at` (used by "recent" ORDER BY ... LIMIT) and `analysis_request.village_id` (used by village history). PostgreSQL does not auto-index foreign keys. Adding `CREATE INDEX ON analysis_request (village_id, created_at DESC)` would serve both queries.

7.4 Write path (point analysis)

```
create_analysis_request()   INSERT analysis_request(point, village_id)   COMMIT; REFRESH -> get id
save_terrain_result()       INSERT terrain_result(request_id, polygon, area, slope)   COMMIT
save_rainfall_result()      INSERT rainfall_result(...)                               COMMIT (if rain)
save_recommendation()       INSERT pond_recommendation(...)                           COMMIT (if rec)
set_cached_response()       SELECT cache row; DELETE if exists; FLUSH; INSERT new row; COMMIT
_record_analysis()          prepend to in-memory list (cap 100) + rewrite JSON file   (always)
```

- Writes happen **after** computation, so the expensive work is never lost due to a DB problem: any DB exception is caught and logged ("Database write skipped") and the user still gets the result.
- Polygon is converted Shapely → WKB with `from_shape(..., srid=4326)`. A malformed polygon is stored as NULL so the numbers are still saved.
- **Write amplification:** one analysis = 4 inserts + cache delete/insert + a JSON file rewrite.

> **BE READY FOR THIS: TRANSACTIONS**
> Each `save_*` call commits separately, so the four inserts are **four transactions, not one**. If the process dies between them, you get an `analysis_request` without its results (not atomic). Fix: do all inserts in one session and call `commit()` once at the end (or use `with db.begin():`) so it is all-or-nothing. Similarly, the cache "delete then insert" can race if two requests write the same key at once (one gets a primary-key violation, which is caught). Fix: PostgreSQL upsert `INSERT … ON CONFLICT (cache_key) DO UPDATE`.

7.5 Read paths

Endpoint	Query	Notes
Cache check	`SELECT … FROM query_cache WHERE cache_key = ?`	PK lookup; TTL checked in Python (age > 7 days = miss).
`GET /api/analyze/{id}`	`SELECT … FROM analysis_request WHERE id = ?`, then the three result rows via relationships	Geometry WKB → Shapely → GeoJSON (`to_shape`, `mapping`). Falls back to the in-memory list, then 404.
`GET /api/villages/{id}/history`	`WHERE village_id = ? ORDER BY created_at DESC`	Newest first.
`GET /api/analyses/recent`	`ORDER BY created_at DESC LIMIT 12`	Dashboard history.

> **N+1 QUERY PROBLEM (GOOD THING TO ADMIT)**
> For history/recent, the code loads N requests, then accesses `r.terrain_result` and `r.recommendation` per row. SQLAlchemy's default lazy loading issues extra queries per row: 1 + 2N queries (25 for 12 rows). Fix: `.options(joinedload(AnalysisRequest.terrain_result), joinedload(AnalysisRequest.recommendation))` to load everything with one JOIN query.

7.6 Read/write ratio and consistency

- **Read-heavy** workload in practice: the same villages are checked repeatedly in demos; history pages are read often. That is why caching gives the biggest win.

- **Consistency:** single database, so reads after writes are strongly consistent. The cache can be stale for up to 7 days, which is acceptable because terrain and 10-year rainfall averages do not change on that timescale.
- **Connection handling:** SQLAlchemy connection pool (default size 5 + overflow 10) with `pool_pre_ping=True` (tests a connection before use, so dropped connections are replaced instead of failing the request).
- **Blocking:** the DB driver (psycopg2) is synchronous, so DB calls inside `async` endpoints briefly block the event loop. Acceptable at this scale (milliseconds); at scale use `asyncpg` / SQLAlchemy async or run in a threadpool.

7.7 File-mode storage details

- History list: newest first, capped at 100, rewritten to JSON after every analysis, loaded at startup (so history survives restarts).
- Cache dict: capped at 500; when full, evict the **oldest inserted** entry (Python dicts keep insertion order) → **FIFO**, not LRU. Lost on restart (not persisted).
- Request id in file mode = current time in ms mod 10^9 (collision possible if two requests in the same ms; a DB sequence avoids this).
- The JSON file is overwritten in place; a crash during the write could corrupt it. Fix: write to a temp file then `os.replace()` (atomic rename).

8. Caching design

Aspect	Choice
What is cached	The **whole consolidated response** of a point analysis (terrain + rainfall + recommendation), not intermediate data.
Key	`f"{round(lat,3)},{round(lon,3)}"`. 0.001° ≈ 111 m, so clicks within the same ~110 m grid cell share one entry.
Pattern	**Cache-aside** (lazy loading): check cache → on miss compute → write to cache.
Store	PostgreSQL table `query_cache` (DB mode) or in-process dict (file mode). The HLD planned Redis; a DB table was chosen to avoid running a second service on a small container. Same idea, different backend.
Expiry	TTL 7 days, checked on read (expired = miss, then overwritten).
Eviction	DB: none (TTL only). Memory: FIFO at 500 entries.
Quality rule	Only cache results with **no degrading warnings**. A result using bundled real archive rainfall is still cached (same data as a live query); a result using the baseline rainfall or a truncated catchment is not, so it is recomputed when services recover.
Effect	Cold point query ≈ 17.5 s (DEM download dominates) → cached ≈ 0.13 s p50; 107 requests/s at 20 concurrent on 1 CPU.
Second-level cache	`data/rainfall_cache.json`: real Open-Meteo 10-year stats for 19 points around Durg–Bhilai, pre-fetched by a script. Used when the live query fails, nearest point within 11 km (matches the ~9–11 km grid of the reanalysis data).
Not cached	Area and contour analyses (inputs are a whole box or file, so hit rates would be low; could hash the file/bounds as a key).

> **TRADE-OFF OF ROUNDING**
> Coarser rounding = higher hit rate but two clicks 100 m apart get the same answer, even though they might be on different sides of a ridge. 3 decimals is close to SRTM's 30 m cell × snapping radius (≈60 m), so the error is small.

Cache invalidation ("the hardest problem"): handled by TTL only. If the algorithm or C coefficient changes, old entries would be wrong; a fix is to include a **model version** in the key (e.g. `"v2:21.218,81.339"`).

9. Concurrency and asynchronous processing

9.1 Async I/O for external calls

- Uvicorn runs one **event loop**. External calls (httpx) are `await`ed, so while waiting for OpenTopography the server serves other requests.
- Terrain and rainfall are started as two tasks in parallel: total wait ≈ max(12 s, 1.25 s) instead of the sum.

9.2 CPU-bound work in worker threads

- Flow routing is CPU-bound. Running it directly in an `async` function would freeze the event loop (health checks would time out and the watchdog would kill the server). So it runs via `asyncio.to_thread`.

- **GIL question:** Python threads cannot run Python bytecode in parallel, but NumPy/Numba/GDAL release the GIL during heavy loops. More importantly the goal is **responsiveness**, not parallel speed: the event loop stays free. Measured: `/api/health` p95 0.42 s while 4 contour jobs were in flight.

9.3 Admission control (`heavy_jobs.py`), the key systems story

```
async def run_heavy(fn, *args):
    waiting += 1
    try:   await wait_for(semaphore.acquire(), timeout=120 s)   # wait for the single slot
    except Timeout: raise ServerBusyError  -> HTTP 503 + Retry-After: 15
    finally: waiting -= 1
    try:   return await asyncio.to_thread(fn, *args)            # run in a worker thread
    finally: semaphore.release(); gc.collect()                  # free big arrays promptly
```

- **Problem found in stress testing:** one contour analysis adds 25–100 MB on top of a ~300 MB server. Two at once exceeded 512 MiB, and the Linux kernel **OOM-killed** the server.
- **Fix:** an `asyncio.Semaphore` with 1 slot (configurable via `BHAGIRATHA_HEAVY_SLOTS`). Heavy jobs run one at a time; others wait in a queue (FIFO-ish). On one CPU, parallel heavy jobs give no extra throughput anyway.
- **Backpressure / load shedding:** if a job waits more than 120 s, reject with **503 + Retry-After** instead of letting the queue grow without limit.
- **Light requests never queue:** health, cache hits, search, history.
- Queue depth is observable (`queue_depth()`).
- Result: after the fix, 0 errors at concurrency 4; peak memory 330 MB. Before: crash at concurrency 2.

9.4 Thread oversubscription (another good story)

The container reports 120 CPUs (`nproc` sees the host), but its cgroup quota allows only 1. NumPy/OpenBLAS/Numba each started ~120 threads fighting for 1 CPU: a contour analysis took 28–49 s. Setting `OMP_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `NUMBA_NUM_THREADS=1` in the start script brought it to 0.8 s. Lesson: size thread pools to the **real CPU quota**, not the visible core count.

9.5 Other concurrency safety

- Each request writes its DEM to a **unique temp file** (`mkstemp`), so concurrent requests cannot overwrite each other's tiles.
- Temp files are deleted in `finally`; the watchdog also removes leftovers older than 30 minutes.
- `flock` in the start script ensures only one server start runs at a time.

10. Resilience and error handling

10.1 Fallback chains (graceful degradation)

Dependency	Primary	Fallback 1	Fallback 2
Rainfall	Live Open-Meteo (12 s deadline)	Bundled real archive data (≤11 km)	Conservative regional baseline (1,150 mm/yr, 920 mm monsoon; deliberately low so ponds are under- not over-sized)
Elevation (point)	SRTM download, one retry on connect error	—	HTTP 502 (cannot invent terrain for a real point)
Elevation (area)	SRTM download	Synthetic terrain + strong warning	Whole-parcel rough estimate if flow analysis fails
Village search	Curated list of 37 villages	Nominatim (3 s timeout)	Curated results only
Database	PostgreSQL + PostGIS	In-memory + JSON file	—

Every fallback adds a message to the response's `warnings` list; the user always knows which data was used.

10.2 Circuit breaker on the rainfall API

```
CLOSED  --(query fails or times out)-->  OPEN  (skip live queries for 600 s,
   ^                                              go straight to fallback)
   |                                                 |
   +----(background probe succeeds)---- probe every 300 s (5-day query, 10 s deadline)
```

- Without it, a blocked host costs a 12 s+ timeout **per request**. With it, only the first failure costs a timeout.
- The background **probe** runs every 5 minutes in its own thread; on success it closes the breaker, on failure it opens it. So user requests never pay the cost of discovering the network is blocked (it plays the role of the "half-open" test).

10.3 Timeouts and retries

- OpenTopography: connect 10 s, read 60 s (tile generation can take ~20 s); retried once only on **connect** errors (safe because nothing was sent yet; idempotent GET).
- Open-Meteo: hard deadline 12 s via `asyncio.wait_for` (per-attempt httpx timeouts can add up to 45 s on a firewalled multi-address host).
- Nominatim: 3 s. DB connect: 3 s.

10.4 Process-level resilience

- **Watchdog** (cron): calls `/api/health` with a 6 s limit; if not HTTP 200, kills the hung process and restarts it. Has a 90 s startup grace period (importing the geospatial stack takes ~20 s). Restart within about 60 s.
- **Warm-up:** PySheds' Numba kernels JIT-compile on first use (~14 s and lots of memory). Compiling inside the server pushed it past 512 MiB, so `start_server.sh` runs `python -m app.warmup` in a **separate short-lived process** when the on-disk kernel cache is cold; the server then only loads cached kernels. The server also warms up on a tiny synthetic DEM in a background thread so the first real user request is fast.

- Log rotation at 20 MB; stale temp files cleaned every run.

11. API design

Method / path	Purpose	Notes
GET /	Single-page Leaflet app	Static files mounted at /static
POST /api/analyze	Evaluate a clicked point	Cached; persisted to DB
GET /api/analyze/{id}	Fetch a past analysis	DB → memory → 404
POST /api/analyze-area	Best site inside a bounding box	422 if min ≥ max
POST /analyzeContour	Best site from KML/KMZ upload	multipart/form-data, field `contour_map`
GET /api/villages/search?q=	Geocoding	Curated + Nominatim; frontend debounces typing
GET /api/villages/{id}/history	Past runs for a village	Newest first
GET /api/analyses/recent	12 most recent runs	Dashboard
GET /api/health	Status, DB mode, uptime, memory (VmRSS), PID	Used by watchdog

Status codes

200 success · **400** wrong file type / unparseable KML · **404** analysis not found · **422** validation error or cannot build grid · **500** unexpected failure · **502** elevation provider failed (upstream/bad gateway) · **503** server busy, with `Retry-After`.

- **Statelessness:** each request carries everything needed; no sessions. (In-memory cache/history are the only process state, see scaling.)
- **Idempotency:** GETs are safe and idempotent. `POST /api/analyze` is not strictly idempotent (it creates a new history record) but the computed result for the same input is the same.
- **Contract-first:** Pydantic response models (`AnalyzeResponse`, `ContourAnalysisResponse`, `AreaAnalysisResponse`) define the JSON shape; `rainfall` and `recommendation` are `Optional` because they can be missing.
- **Not implemented:** authentication, rate limiting, API versioning (/v1). Justification: read-only analysis tool on the campus network with no personal data.

12. Deployment and operations

- Campus container `stu72_sys3`: cgroup limit 1 CPU, 512 MiB RAM; public at `http://10.1.75.53:3247/` (container port 3000 mapped).
- `scripts/start_server.sh`: sets PYTHONPATH and thread-pool env vars → refuses to start a second copy (pgrep + `flock`) → pre-compiles Numba kernels if needed → rotates logs → starts Uvicorn with `nohup` in the background → checks the process is alive.
- `scripts/watchdog.sh`: run by cron; cleans temp files; health check; restart if unhealthy.
- Config by environment variables / `.env` (12-factor style): `OPENTOPOGRAPHY_API_KEY`, `DATABASE_URL`, `BHAGIRATHA_HEAVY_SLOTS`, `BHAGIRATHA_QUEUE_TIMEOUT_S`.
- PostgreSQL is not on the container; it can be reached by SSH reverse tunnel (`ssh -R 5433:localhost:5433 …`); otherwise file mode.
- Version control: Git + GitHub; no CI pipeline (tests run manually before deploy).

- Single Uvicorn worker on purpose: more workers would multiply memory (each ~300 MB) beyond 512 MiB.

13. Performance and stress testing

Run on the evaluation server with the load generator on the **same container** (it competes for the single CPU, so numbers are pessimistic). Idle memory 290.5 MB, peak 329.7 MB.

Scenario	Concurrency	Requests	Errors	p50	p95	Throughput
/api/health	20	200	0	0.087 s	0.279 s	163 req/s
Point analysis (cached)	20	200	0	0.133 s	0.420 s	107 req/s
Point analysis (cold, first call)	1	1	0	17.5 s		—
Contour (6.7 MB KML)	1	4	0	0.84 s	1.07 s	1.12 req/s
Contour	2	4	0	1.69 s	1.76 s	1.16 req/s
Contour	4	4	0	7.10 s	9.95 s	0.40 req/s
Health during 4× contour	1	29	0	0.070 s	0.418 s	—

- Total 446 requests (incl. warm-up), 0 errors.
- **Reading the table:** contour throughput stays ~1.1 req/s from concurrency 1 to 2 because jobs are serialised; latency grows with queue position (that is queueing, by design). The concurrency-4 slowdown is mostly the load generator itself sharing the one CPU.
- Health stays fast under heavy load → event loop is not blocked.
- **Little's Law** (if asked): $L = \lambda \cdot W$. With one slot and ~0.85 s per job, max sustainable throughput ≈ 1.2 contour jobs/s; beyond that the queue grows until the 120 s timeout returns 503.
- **Bottlenecks:** cold point query = network (DEM download ~12 s); contour = CPU (interpolation 0.72 s is the largest stage); memory is the hard limit on concurrency.

14. Testing strategy

28 automated pytest tests in 7 files, plus the stress-test script.

File	What it checks
test_terrain.py	D8 + depression filling on a synthetic bowl-shaped DEM (known answer).
test_contour.py	Real 1 m contour file: parser, full grid, full pipeline finds a site, HTTP end-to-end, rejects non-KML.
test_extended.py	Real KMZ, empty file, malformed XML, area endpoint, search, health.
test_scientific.py	Rational-method units and 70% capture; depth bounded and monotonic; catchment score continuous; int16 SRTM DEM works; pond stays inside parcel; GeoTIFF pixel centres match samples; circuit breaker; bundled vs baseline rainfall; **heavy jobs never overlap and the queue times out**.
test_rainfall.py	Aggregation maths, null days, malformed response.
test_cache.py	Miss then hit; TTL expiry treated as miss.
test_db.py	Full round-trip into PostGIS and back.

Kinds of tests: **unit** (formulas, aggregation), **property-style** (monotonic depth, continuous score), **integration** (DB round-trip, real files), **end-to-end** (HTTP via TestClient), **failure-injection** (monkeypatched network failures for the circuit breaker), **load** (stress script). Network-dependent pieces are tested with synthetic data so the maths is verified offline.

15. Design patterns used

Pattern	Where
Modular monolith / layered architecture	API layer → domain engines → persistence
Adapter	KML contours and SRTM tiles both adapted to one float32 GeoTIFF interface
Circuit breaker	Open-Meteo calls (`_last_failure_at` , 600 s cooldown, background probe)
Fallback chain / graceful degradation	Rainfall live → bundled → baseline; DB → file mode
Cache-aside	Point analysis response cache
Dependency injection	FastAPI `Depends(_get_db_lazy)` provides the DB session (or None)
Repository / DAO	`crud.py` hides SQL from `main.py`
Bulkhead / admission control / backpressure	Semaphore for heavy jobs, 503 + Retry-After
Fan-out / scatter-gather	Terrain and rainfall tasks in parallel
Data Transfer Objects	Pydantic schemas
Health check + supervisor	`/api/health` + watchdog

16. Weak spots and how you would fix them

Examiners like it when you know your own system's limits. Admit them confidently and give the fix.

Weakness	Fix
Four separate commits per analysis (not atomic)	One transaction: add all rows, commit once.
Cache write = SELECT + DELETE + INSERT (race on the same key)	`INSERT … ON CONFLICT DO UPDATE` (upsert).
N+1 queries in history endpoints	`joinedload` / explicit JOIN.
No index on `village_id`, `created_at`	Composite index `(village_id, created_at DESC)` + index on `created_at`.
Sync DB driver inside async endpoints	asyncpg / SQLAlchemy async, or run DB calls in a threadpool.
In-memory cache & history are per-process	Move to Redis / PostgreSQL so multiple workers share state.
JSON history file not written atomically	Write temp file then `os.replace`.
FIFO eviction	LRU (`OrderedDict.move_to_end` on hit) or Redis `allkeys-lru`.
Time-based request id in file mode can collide	DB sequence / UUID.
No auth, no rate limiting	API keys or OAuth for admins; token-bucket rate limit at a reverse proxy.
Constant runoff coefficient C = 0.35	Look up C per land-cover/soil class (e.g. from ESA WorldCover + soil maps).
"Best site" = max accumulation only (can land in a river)	Mask out existing water bodies (OSM/water-body layer), add land-use, soil, ownership constraints.
Cache key ignores model version	Prefix key with algorithm version.
No CI	GitHub Actions running pytest on every push.

17. Scaling to production (likely "what if 10,000 users?" question)

1. **Separate light and heavy work.** API servers stay stateless and fast; heavy terrain jobs go to a **job queue** (Celery/RQ with Redis or RabbitMQ). API returns `202 Accepted` + job id; client polls `GET /jobs/{id}` or gets a WebSocket/SSE update. Workers scale independently (autoscale on queue length).
2. **Horizontal scaling behind a load balancer** (Nginx / cloud LB, round-robin or least-connections). Works only once all state (cache, history) is moved out of process.
3. **Shared cache:** Redis with TTL and LRU eviction; a CDN for static frontend files and map tiles.
4. **Pre-fetch / own the DEM:** download SRTM tiles for the target states once into object storage (S3/MinIO) instead of calling OpenTopography per request; this removes the 12 s network step and the rate limit. Same for rainfall: precompute a gridded rainfall table.
5. **Database:** connection pooling (PgBouncer), read replicas for history/dashboard reads, partitioning `analysis_request` by time, the indexes listed above.

6. **Containerise** with Docker; orchestrate with Kubernetes (health checks replace the watchdog, resource limits per pod).
7. **Observability:** structured logs, metrics (latency, queue depth, cache hit rate), tracing; alerts on error rate.
8. **Security:** HTTPS, authentication for administrators, rate limiting, upload size limits, XML hardening (lxml with entity resolution disabled against XXE/billion-laughs).

> **LOAD BALANCING IN THIS PROJECT**
> Not used, deliberately: on one CPU and 512 MiB, a second instance would compete for the same resources and roughly double memory. Admission control is the effective load management here.

18. Viva questions and answers

A. Overview and architecture

> **Q1. Explain your project in two minutes.**
> Use the cover pitch: inputs (point / area / contour), terrain pipeline (DEM → Priority-Flood → D8 → accumulation → site → catchment), rainfall (Open-Meteo 10 yr), Q = C·I·A, sizing and score, map overlay; then the system parts: FastAPI modular monolith, PostGIS + cache, async, admission control, fallbacks, 1-CPU deployment, stress test.

> **Q2. Why a monolith? Isn't microservices better?**
> Not here. The steps of one request are tightly coupled and exchange large rasters; network hops would add latency and serialisation cost. One container with one CPU means extra services compete for the same resources. The module boundaries are clean (typed interfaces), so Terrain could be split out as a worker service later.

> **Q3. What is the critical path of a point request?**
> The DEM download (~12 s) followed by flow routing. Rainfall (~1.25 s) runs in parallel so it is off the critical path. A cache hit skips everything.

> **Q4. Where is the "AI" in this project?**
> It is a rule-based decision-support system built on established hydrology algorithms, not a trained ML model. The "intelligence" is automatic site discovery from terrain plus a multi-criteria weighted scoring model. A learned model would need labelled data (successful/failed ponds), which we do not have; that is future work.

> **Q5. How do modules communicate?**
> In-process function calls with Pydantic objects as contracts: Terrain returns `TerrainResult`, Rainfall returns `RainfallResult`, Recommendation takes both and returns `RecommendationResult`. No shared global state between them.

B. Algorithms

> **Q6. What is D8 and what are its limitations?**
> Each cell sends all its water to the single steepest-descent neighbour out of 8. Simple, O(N), standard. Limitation: water cannot split between cells (no divergent flow), so it produces parallel artificial lines on planar slopes. Alternatives: D-infinity (Tarboton) or MFD (multiple-flow-direction). D8 is fine for finding the main convergence point.

> **Q7. Why fill depressions first?**
> Small pits in the DEM (sensor noise) would trap flow so it never reaches the outlet; accumulation and catchments would be broken into fragments. Priority-Flood raises each pit to its spill elevation in O(N log N).

> **Q8. Explain Priority-Flood.**
> Put all border cells in a min-heap. Repeatedly pop the lowest cell; for each unvisited neighbour, set its elevation to max(its own, popped cell's), mark it visited and push it. Like water rising from the edges. Each cell enters the heap once → O(N log N).

Q9. Why is the latitude correction needed?

The raster is in degrees. A degree of longitude is cos(φ) times a degree of latitude (0.934 at 21°N). Without converting to metres, D8 would misjudge east–west slopes by ~7%. The code builds a metric grid with $\Delta x = \Delta\lambda \cdot 111{,}320 \cdot \cos\varphi$.

Q10. How do you choose the pond site automatically?

Max flow accumulation in the interior (excluding an 8% border where edge effects inflate accumulation); in area mode, also restricted to the drawn parcel while flow is routed over a bigger tile.

Q11. What is pour-point snapping?

A clicked point is rarely on the exact one-cell D8 channel; one cell off gives a tiny catchment. We move it to the max-accumulation cell within 2 cells (~60 m).

Q12. How is the catchment computed?

Reverse graph traversal from the pour cell: include all cells whose flow direction points into an included cell. Area = cell count × metric cell area. Then polygonize into GeoJSON.

Q13. How do you turn contour lines into a DEM?

Contour vertices become scattered elevation samples; decimate to ≤ ~20k points by fixed stride; SciPy griddata linear interpolation on a Delaunay triangulation onto a 200-cell grid; nearest-neighbour fill outside the convex hull; write a GeoTIFF.

Q14. Why linear, not cubic interpolation? Why stride, not random sampling?

Cubic overshoots at data edges, creating fake pits and peaks that distort flow. Stride sampling is deterministic (same input → same output, reproducible tests) and keeps spatial spread.

Q15. Explain the Rational Method and your units.

$Q = C \cdot I \cdot A$. We use the volumetric form: I is seasonal rainfall depth in metres, A in m^2, so Q is in m^3 of seasonal runoff. Classical form uses intensity (mm/h) to get peak discharge (m^3/s) for drain design. C = 0.35 for cultivated loamy land.

Q16. Why only 70% capture? Why 2.5–4.5 m depth?

Capturing all runoff would need an oversized pond and expose it to extreme years; the remainder spills. Below 2.5 m evaporation losses are proportionally high; above 4.5 m unlined earthen embankments become unsafe. Depth scales linearly between.

Q17. How did you choose the score weights?

Expert-judgement weights (multi-criteria analysis): catchment adequacy matters most (40%) because it determines whether the pond fills, slope (35%) drives cost and safety, rainfall (25%). Each sub-score is piecewise-linear and continuous so small input changes cannot cause jumps. Weights are tunable; with field data they could be calibrated.

Q18. What's the time complexity of the whole pipeline?

Dominated by O(n log n) Delaunay (contour mode) and O(N log N) Priority-Flood; D8, accumulation, argmax are O(N). Measured <0.2 s for 3×10^4 cells.

C. Database

> **Q19. Describe your schema.**
> village, analysis_request (POINT, FK village), and three 1:1 result tables (terrain_result with polygon, rainfall_result, pond_recommendation) whose request_id is both PK and FK; plus query_cache keyed by rounded coordinates. SRID 4326.

> **Q20. What exactly happens on a write?**
> INSERT analysis_request → commit → refresh to get the generated id → INSERT terrain, rainfall, recommendation (each commits) → cache upsert (delete+insert) → append to history JSON. Writes happen after computing, and DB errors are caught so the user still gets a result.

> **Q21. Is your write atomic? What if it crashes in the middle?**
> No; each insert commits separately, so a crash could leave a request without results. Fix: one transaction with a single commit (ACID atomicity). Readers handle missing results already (they return None for missing parts).

> **Q22. What indexes do you have? What would you add?**
> B-tree PK indexes on all keys (cache and by-id lookups are O(log n)); GiST spatial indexes on geometry columns (GeoAlchemy2 default). I'd add (village_id, created_at DESC) and created_at indexes for history and recent queries.

> **Q23. What is a GiST index?**
> Generalized Search Tree; for geometries it works like an R-tree over bounding boxes. A spatial query first filters by bounding-box overlap using the index, then checks exact geometry. Makes "within distance" / "intersects" queries fast instead of a full scan.

> **Q24. What is SRID 4326?**
> WGS84 latitude/longitude (what GPS and GeoJSON use). For accurate areas in m² you'd cast to geography or project to UTM (EPSG:32644 for 81°E); our area is computed from metric cell sizes instead.

> **Q25. Where is the N+1 problem in your code?**
> History/recent endpoints load requests, then lazily load terrain and recommendation per row → 1 + 2N queries. Fix with joinedload / JOIN.

> **Q26. What if the database is down?**
> Detected at startup with a 3 s SELECT 1; server switches to file mode: in-memory cache + history list persisted to JSON. All endpoints still work. Runtime DB errors are caught per request. pool_pre_ping replaces dead connections.

> **Q27. Why store seasonal rainfall as JSON?**
> Small, always read together, never filtered on; a JSON column keeps the schema flexible (e.g. adding monthly values later). It's a conscious denormalisation.

> **Q28. SQL vs NoSQL for this project?**
> SQL + PostGIS: spatial types and indexes, joins, foreign keys, transactions. Data is structured and relational. NoSQL would help only at huge write scale or with very unstructured data.

Q29. How do you prevent SQL injection?
All queries go through SQLAlchemy ORM with bound parameters; no string-built SQL. Inputs are also type-validated by Pydantic.

D. Caching

Q30. Explain your cache.
Cache-aside; key = lat/lon rounded to 3 decimals (~110 m cell); value = full response; 7-day TTL; stored in PostgreSQL or memory; only full-quality results cached. Cold 17.5 s → cached 0.13 s.

Q31. Why not Redis?
HLD planned Redis; at this scale a DB table avoids running another service on a 512 MiB container. Redis would be the choice at scale (shared across workers, LRU eviction, sub-ms).

Q32. How do you invalidate the cache?
TTL only, because the data (terrain, 10-year averages) is effectively static. Degraded results are never cached. If the model changes, add a version prefix to the key.

Q33. What is your eviction policy?
In memory: FIFO at 500 entries. DB: no size limit, TTL on read. LRU would be better for skewed popularity.

Q34. What are cache stampede / thundering herd? Do you handle it?
Many simultaneous misses for the same key all compute the same thing. Not explicitly handled; the heavy-job semaphore limits the damage (they queue). Fix: single-flight / per-key lock so only one computes and others await it.

E. Concurrency, performance, resilience

Q35. How does async help here?
External calls are I/O-bound; awaiting them frees the event loop to serve other requests, and terrain and rainfall run concurrently.

Q36. Why run flow routing in a thread if Python has a GIL?
To keep the event loop responsive (health checks, cache hits). NumPy/Numba release the GIL in their inner loops. The aim is responsiveness, not parallel speed-up (one CPU anyway).

Q37. What was the OOM problem and how did you solve it?
Two concurrent contour jobs pushed memory past 512 MiB and the kernel killed the server. Solution: admission control, a 1-slot semaphore; extra jobs queue; after 120 s they get 503 + Retry-After; gc.collect after each job; Numba compilation moved to a separate pre-start process.

Q38. Why 503 and not just wait forever?
Unbounded queues grow memory and latency without limit and clients time out anyway. Fail fast with a retry hint (backpressure) so clients can retry later.

Q39. Explain your circuit breaker.
After an Open-Meteo failure, skip live calls for 600 s and use the fallback directly; a background probe every 300 s resets it when Open-Meteo answers. One timeout instead of one per request.

Q40. What happens when Open-Meteo is blocked (as on campus)?
Bundled real archive data for 19 points near Durg–Bhilai (within 11 km) gives identical numbers to a live query; elsewhere the conservative baseline. The user sees a warning.

Q41. Why is the baseline deliberately low?
Fail-safe: underestimating rainfall under-sizes the pond (wasted potential) rather than over-sizing it (wasted money, dry pond).

Q42. What was the thread-oversubscription issue?
nproc showed 120 cores but cgroup allowed 1; numeric libraries started ~120 threads each → 28–49 s per job. Pinning thread pools to 1 → 0.8 s.

Q43. How do you ensure high availability?
Watchdog health check + auto-restart (~60 s), fallbacks for every external dependency, DB-less mode, admission control to prevent crashes. True HA would need multiple instances behind a load balancer.

Q44. What is p95 latency and why report it?
95% of requests are faster than this value. Averages hide slow tail requests that users notice; percentiles show the tail.

F. API, frontend, security

Q45. Why REST? Why POST for analysis?
Simple, cacheable, documented by OpenAPI. POST because the request has a body, creates a record, and file upload needs multipart.

Q46. How is input validated?
Pydantic types and ranges (422), explicit bounds check, file extension check (400), parser errors (400), grid errors (422).

Q47. How does the frontend draw results?
Leaflet: Esri satellite basemap, GeoJSON layer for the catchment, a marker for the pond with popup, dashed rectangle for the parcel; Chart.js for rainfall; village search input is debounced (setTimeout) to avoid a request per keystroke.

Q48. What security risks exist?
No auth or rate limiting; large uploads (no explicit size limit) could exhaust memory; XML parsing of untrusted KML (XXE / entity expansion); CORS not configured. Fixes: auth, rate limits, max upload size, hardened XML parser, HTTPS.

Q49. Why is the API key not in the code?
Secrets come from environment variables / .env (not committed; .env.example is the template), following 12-factor config.

G. Testing and evaluation

> **Q50. How do you know your results are correct without field data?**
> Synthetic DEM with a known answer; consistency checks (polygon area vs cell area 3.9114 vs 3.9116 km², valid polygon, not touching edge); hand calculation of Q matches output; property tests (monotonic depth, continuous score, pond inside parcel).

> **Q51. How did you stress test?**
> A script on the server: 200 health requests and 200 cached point requests at 20 concurrency, contour at concurrency 1/2/4, and health latency during 4 contour jobs; sampled RSS memory. 0 errors, peak 330 MB.

> **Q52. What would you improve with more time?**
> Job queue + workers, Redis, pre-fetched DEM tiles, single-transaction writes and upserts, indexes, land-use/soil-based C, masking existing water bodies, auth, CI pipeline.

H. Curveball questions

> **Q53. Your sample site is in a river. Isn't the system wrong?**
> It does exactly what it is defined to do: find maximum convergence, and rivers are where water converges most. It shows why field verification and extra constraints (masking water bodies, land use) are required; this is stated as a limitation.

> **Q54. What if two users click the same point at the same time?**
> Both miss the cache, both compute (serialised by the heavy-job slot), both insert separate history rows; the second cache write replaces the first (or hits a caught PK conflict). Results are identical, so correctness is fine; just duplicate work (fix: single-flight).

> **Q55. How would you support multi-village batch planning?**
> Batch endpoint that enqueues one job per village into a queue; workers process them; results stored and fetched by batch id. Shared DEM tiles cached in object storage.

> **Q56. CAP theorem: where does your system sit?**
> Single-node PostgreSQL → consistent; no partitions in a single node. Overall the system favours availability: when the DB or an API is unreachable it still answers with degraded (clearly flagged) data.

> **Q57. Why does the catchment of the area mode extend outside the parcel?**
> Water that reaches the pond comes from all upstream land, not only the parcel. Flow is routed over a larger tile; only the pond location is restricted to the parcel.

19. Numbers cheat sheet

Model constants	
Runoff coefficient C	0.35
Capture fraction	70%
Depth range	2.5–4.5 m (5k–200k m^3)
Score weights	40 / 35 / 25, clamp [5, 98]
Ideal slope / catchment	≤ 4% / 0.5–5 km^2
Monsoon months	June–September
Rainfall history	10 full years
Baseline rainfall	1,150 mm/yr, 920 monsoon
Bundled rainfall	19 points, ≤ 11 km
DEM	SRTM GL1 30 m; tile ±0.05°, retry ±0.12°
Snap radius	2 cells ≈ 60 m
Border exclusion	8%
Contour grid / max points	200 cells / 20,000
m per degree	111,320 (× cos φ for lon)

System constants and results	
Container	1 CPU, 512 MiB
Memory idle / peak	290 / 330 MB
Heavy slots / queue timeout	1 / 120 s → 503, Retry-After 15
Cache key / TTL	3 decimals (~110 m) / 7 days
Memory cache / history cap	500 / 100
Rain deadline / breaker	12 s / 600 s; probe 300 s
DEM timeouts	connect 10 s, read 60 s, 1 retry
DB connect check	3 s
Watchdog grace	90 s
Cold vs cached point	17.5 s vs 0.13 s
Contour job	≈ 0.84 s
Throughput (cached)	107 req/s @ 20 conc.
Stress test	446 requests, 0 errors
Tests	28 pytest tests

Sample-map result

1,355 contours · 267–298 m · site 21.24171° N, 81.28692° E · A = 3.9116 km^2 · slope 3.99% · rain 1,415.2 / 1,251.4 mm · **Q = 1,713,242 m^3** · V = 1,199,269 m^3 · depth 4.5 m · area 266,504 m^2 · score 98.

Five phrases to drop in naturally

- "Modular monolith with typed interfaces, so it can be split later."
- "Cache-aside keyed on a rounded spatial grid, only full-quality results are cached."
- "Admission control with a semaphore and 503 backpressure stopped OOM kills."
- "Fallback chain plus circuit breaker: graceful degradation, always reported to the user."
- "Priority-Flood O(N log N), latitude-corrected D8 O(N)."

> **VIVA TIP**
> When you don't know an answer, explain how you would find out or test it. When they point out a weakness, agree, say why it is acceptable at this scale, and give the production fix. That shows system-design thinking.